Example: how to use these regular expression properties and language operators?

- *L = { w | w is a binary string which does not contain two consecutive 0s or two consecutive 1s anywhere)*
  - E.g., w = 01010101 is in L, while w = 10010 is not in L
- Goal: Build a regular expression for L
- Four cases for w:
  - Case A: w starts with 0 and |w| is even
  - Case B: w starts with 1 and |w| is even
  - Case C: w starts with 0 and |w| is odd
  - Case D: w starts with 1 and |w| is odd
- Regular expression for the four cases:
  - Case A: $(01)^*$
  - Case B: $(10)^*$
  - Case C: $0(10)^*$
  - Case D: $1(01)^*$
- Since L is the union of all 4 cases:
  - Reg Exp for L = $(01)^* + (10)^* + 0(10)^* + 1(01)^*$

Give more strings in L aside from w = 01010101

Given the regular expression (BN01 section):

RE = 1*00*1(0+1)*

Task:

1. Construct one full valid string that matches the regular expression by explicitly applying each operators ({*},{+}).
   - Indicate clearly which parts of the regular expression.
2. Draw the DFA that accepts the same string:
   - Label all states and transitions,
   - Identify the start and accepting state(s),
   - Make sure the DFA accepts the constructed string in Question #1.

Interesting Part!

Machine Code Translation Task

Based on the sample CFG production rules presented in the project, consider the context-free language that generates the following variable declaration and assignment:

```
x: int
x = 100
```

Task: Write the equivalent machine-level code for this statement using an assembly language of your choice (e.g., `x86`, `MIPS`, or `ARM`), based on your understanding of instruction-level representation from your *Computer Architecture and Organization* course.